LAPORAN AUDIT APLIKASI

Sistem Pembuatan Dokumen Buku Kerja Asesor V.2023

Audit kode, keamanan, keandalan, dan pengalaman pengguna (UI/UX)

Keterangan	Isi
Objek audit	Repositori skkni-extractor (FastAPI + Jinja2 + PyMuPDF)
Versi aplikasi	1.0.0 — commit b2a12a5 (branch main)
Lingkungan uji	Linux, Python 3.14, Chromium 1280×577 px & 390×844 px
Tanggal audit	26 Oktober 2026
Sifat audit	Read-only. Tidak ada perubahan kode aplikasi.
Penerima laporan	Tim programmer (pelaksana perbaikan)
Metode	Pembacaan kode, eksekusi test suite, pengukuran DOM/CSS langsung di peramban

Nilai keseluruhan: 5,3 / 10 — Fungsional dan stabil, tetapi tampilan dan kontrol akses belum siap dipakai orang lain.

1. Ringkasan Eksekutif

Aplikasi ini secara fungsi berjalan: seluruh halaman terbuka, 26 test otomatis lulus, alur unggah PDF → ekstraksi unit → susun MUK → cetak/unduh 25 dokumen Buku Kerja dapat diselesaikan tanpa error. Fondasi teknisnya justru bagian yang paling sehat: penulisan berkas atomik, token CSRF di semua POST, validasi unggahan, penanganan galat tanpa membocorkan traceback, dan kueri SQL yang berparameter.

Masalah utamanya ada di tiga tempat. Pertama, tampilan: aplikasi memakai satu sistem desain datar untuk 32 halaman, dengan 1.416 atribut gaya inline, teks sekunder berkontras di bawah ambang WCAG AA, tombol aksi setinggi 24–27 px, dan satu halaman (Pengaturan MUK) berisi 88 kontrol dalam 9 tab tanpa penanda "belum disimpan". Kedua, kontrol akses: login ada, tetapi dari 13 endpoint POST hanya satu yang memeriksa peran — sehingga unggah dokumen, simpan MUK, koreksi unit, dan pembuatan dokumen dapat dilakukan siapa pun yang bisa membuka alamat aplikasi. Ketiga, pengujian: dari 29 berkas test, hanya 3 yang berbentuk unittest, sehingga perintah penemuan otomatis hanya menjalankan 26 test dari sekitar 28 berkas.

Untuk pemakaian pribadi di satu komputer, kondisi ini masih dapat diterima. Untuk dipakai beberapa asesor dalam satu jaringan LSP, dua hal wajib dibereskan lebih dulu: penegakan hak akses dan penggantian kredensial bawaan.

Skor per area

Area	Skor	Ringkas	Utama
Fungsi & alur bisnis	8,0	Alur 25 dokumen BNSP lengkap dan konsisten	Pertahankan
Keandalan backend	7,0	Atomik, tervalidasi, tanpa traceback bocor	Aktifkan WAL, kurangi baca ulang
Kualitas kode	5,5	Berjalan, tetapi monolit dan banyak duplikasi	Pecah main.py & template
UI / UX	5,0	Rapi tetapi datar, mikro-teks, kontrol kecil	Sistem desain + aksesibilitas
Keamanan	4,0	CSRF baik, RBAC nyaris tidak dipakai	Tegakkan peran, hapus kredensial bawaan
Pengujian	4,0	Test lulus, tetapi tidak dapat diotomatiskan	Migrasi ke pytest + CI
Dokumentasi	6,0	Tebal dan rinci, sebagian sudah usang	Selaraskan status & klaim

Skala: 1–3 belum layak, 4–5 perlu perbaikan besar, 6–7 layak dengan catatan, 8–10 baik.

Kesimpulan penilaian UI/UX: aplikasi ini tidak "jelek" secara teknis — CSS-nya bersih, variabel warna konsisten, dan layout tidak rusak. Kesan kurang menarik muncul dari ketiadaan hierarki: semua kartu berbobot sama, semua aksi tampil sebagai tombol abu-abu kecil, tidak ada satu pun elemen yang menuntun mata ke "langkah berikutnya".

2. Cakupan dan Metode Audit

Yang diperiksa:

- Seluruh kode Python di app/ (10.771 baris) dan seluruh template Jinja2 (8.082 baris).
- Halaman yang dijalankan langsung: dashboard, daftar & detail SKKNI, daftar & detail skema, Pengaturan MUK, halaman koreksi unit, halaman cetak.
- Pengukuran di peramban: kontras warna, ukuran font, tinggi target klik, luapan horizontal pada lebar 390 px, jumlah atribut gaya inline, jumlah label form.
- Eksekusi test suite (unittest discover) dan pemeriksaan impor aplikasi.

- Pembacaan requirements.txt, run.sh, docs/, dan riwayat commit.

Yang tidak diperiksa: akurasi isi PDF hasil ekstraksi per unit (butuh pembandingan manual dokumen per dokumen), dan pengujian di Windows/macOS.

3. Temuan UI/UX (fokus utama)

Angka pada bagian ini diukur langsung dari DOM halaman yang berjalan, bukan perkiraan.

3.1 Masalah yang paling mengganggu

UX-01. Tidak ada hierarki visual — semua terlihat sama penting [P1]

Bukti: Dashboard menampilkan 4 kartu statistik berbobot identik, lalu 2 kartu status, lalu tabel aktivitas; tidak ada judul bagian yang menonjol, tidak ada warna aksen, dan tidak ada tombol tindakan utama. Di halaman detail skema terdapat 67 tombol/tautan, sebagian besar bergaya "btn-small" abu-abu dengan tinggi 27 px.

Dampak: Mata pengguna tidak tahu harus mulai dari mana. Asesor baru akan menebak-nebak langkah (unggah? ekstrak? susun MUK? cetak?), padahal urutannya sebenarnya baku.

Rekomendasi: Tetapkan satu "aksi utama" per halaman dengan warna aksen tunggal (mis. biru tua), sisanya sekunder. Tambahkan blok "Langkah berikutnya" di dashboard yang mengarahkan ke alur: unggah SKKNI → unggah skema → ekstrak unit → susun MUK → cetak. Beri judul bagian yang berukuran lebih besar dari isi kartu.

UX-02. Kontras teks sekunder gagal standar keterbacaan [P1]

Bukti: Terukur di peramban: teks .muted dan footer #6b7a8c di atas putih = 4,02:1; .stat-label, .stat-note, .status-detail, dan header tabel = 4,39:1 pada ukuran 11,5–12,5 px; badge hijau #1f9d55 di atas #e7f6ee = 3,49:1. Ambang WCAG AA untuk teks normal adalah 4,5:1.

Dampak: Keterangan status, satuan, dan tanggal sulit dibaca — terutama di proyektor saat presentasi atau di laptop dengan layar redup. Informasi penting seperti "3 dari 4 unit kompetensi" justru berada di lapisan kontras terendah.

Rekomendasi: Gelapkan --muted menjadi sekitar #4a5a6b (kontras ±6:1) dan gelapkan teks badge (mis. #157a41 di atas latar badge) sampai minimal 4,5:1. Perbesar teks mikro 11,5 px menjadi minimal 12,5 px.

UX-03. Halaman Pengaturan MUK terlalu padat untuk satu layar [P1]

Bukti: skema_muk.html berukuran 1.365 baris dengan 353 atribut style inline, 88 kontrol input/select, dan 9 tab (Personel, Kelompok Pekerjaan, IA.02 … IA.08). Seluruh isi berada dalam satu form; penyimpanan dilakukan lewat satu tombol "Simpan Seluruh Pengaturan".

Dampak: Pengguna harus menavigasi 9 tab untuk satu pekerjaan. Karena tidak ada penanda "belum disimpan", menutup tab atau menekan Kembali menghapus seluruh isian tanpa peringatan. Ini risiko kehilangan data yang paling mahal di aplikasi ini.

Rekomendasi: Tambahkan: (a) penanda per tab berisi perubahan belum tersimpan, (b) peringatan beforeunload bila ada perubahan, (c) tombol simpan per tab di samping simpan-semua, (d) validasi wajib sebelum kirim dengan pesan per field. Jangka menengah: pecah menjadi sub-halaman per instrumen dengan URL sendiri agar bisa di-bookmark.

UX-04. Tidak ada indikator proses pada operasi yang lama [P1]

Bukti: Tidak ditemukan satu pun elemen loading/spinner/progres di seluruh template (pencarian kata "loading", "memuat", "Memproses" = 0 hasil). Pembuatan dokumen Word 25 formulir, unggah PDF 50 MB, dan ekstraksi awal semuanya berjalan sebagai POST biasa yang menunggu sampai selesai. Hanya job ekstraksi latar belakang yang punya bilah progres.

Dampak: Peramban tampak "hang". Pengguna cenderung menekan tombol dua kali, sehingga muncul permintaan ganda atau unggahan ganda.

Rekomendasi: Nonaktifkan tombol saat dikirim + tampilkan teks "Memproses…" dan indikator tak-tentu. Untuk pembuatan dokumen, pindahkan ke job manager yang sudah ada (app/jobs.py) agar seragam dengan ekstraksi.

3.2 Aksesibilitas dan konsistensi

UX-05. Formulir hampir tanpa label yang benar [P2]

Bukti: Ada 102 kontrol (87 input + 15 select) tetapi hanya 50 elemen <label>, dan hanya 1 yang memakai atribut for=. Di seluruh template hanya ada 3 atribut aria-*/role=.

Dampak: Pembaca layar tidak dapat menyebutkan nama field; pengguna keyboard tidak tahu fokus ada di mana. Ini juga menyulitkan pengisian cepat karena klik pada teks label tidak memfokuskan field.

Rekomendasi: Hubungkan setiap label dengan id field (for/id), tambahkan aria-label pada kontrol tanpa label visual, dan pastikan urutan tab mengikuti urutan baca. Tambahkan :focus-visible yang jelas — saat ini hanya .filter yang punya outline fokus.

UX-06. Target klik terlalu kecil [P2]

Bukti: Terukur: tombol "Lihat" dan "Koreksi" di tabel setinggi 27 px; tombol "Masuk"/"Keluar" 24 px; sebagian besar aksi sekunder 27 px. Rekomendasi umum adalah 44×44 px (WCAG 2.5.5) atau minimal 32 px untuk tampilan desktop padat.

Dampak: Salah klik pada tabel unit — terutama di layar sentuh — dapat memicu aksi yang salah (mis. membuka koreksi padahal ingin melihat detail).

Rekomendasi: Naikkan tinggi tombol tabel menjadi 32–36 px dengan padding vertikal 8 px, dan jarak antar tombol minimal 8 px. Untuk baris tabel yang padat, gunakan satu tombol "Buka" dan pindahkan aksi lain ke menu titik-tiga.

UX-07. Navigasi tidak nyaman di layar kecil [P2]

Bukti: Pada lebar 390 px, menu samping berubah menjadi bilah horizontal selebar 946 px sehingga harus digeser ke samping; tidak ada tombol menu ringkas. Tabel berubah menjadi blok yang menggulir horizontal dengan white-space: nowrap.

Dampak: Aplikasi tidak nyaman dipakai dari ponsel/tablet — padahal asesor sering bekerja di lapangan dengan tablet.

Rekomendasi: Ganti bilah horizontal dengan tombol hamburger + panel geser. Untuk tabel, sembunyikan kolom sekunder di layar kecil dan tampilkan bentuk kartu (label–nilai) agar tidak perlu menggulir dua arah.

UX-08. Tiga sistem gaya berbeda dalam satu aplikasi [P2]

Bukti: Halaman aplikasi memakai style.css (368 baris) + 1.416 atribut style inline di template; halaman cetak cetak_apl.html memakai 903 baris CSS sendiri di dalam <style>; halaman login juga menambah gaya inline. Warna aksen muncul dalam tiga varian: --navy (#0b2540), #0284c7, dan #16a34a.

Dampak: Perubahan tampilan harus dikerjakan di tiga tempat. Inilah sebab utama tampilan terasa "tambal sulam": tombol hijau terang di halaman cetak, biru tua di halaman aplikasi, dan abu-abu di tabel.

Rekomendasi: Jadikan style.css satu-satunya sumber: pindahkan CSS cetak ke berkas terpisah (print.css) yang hanya mengatur @media print, dan hapus seluruh atribut style inline. Tambahkan kelas komponen (btn-primary, btn-secondary, btn-danger, badge) agar tidak ada lagi warna hard-code seperti #16a34a dan #fee2e2 di template.

UX-09. Istilah teknis bocor ke antarmuka asesor [P3]

Bukti: Halaman detail skema menampilkan "NAMA INTERNAL: data/schema_library/docx/ad086...docx", "SHA-256: 81088860f00cfd34...", dan "identifikasi: lengkap". Dashboard menampilkan nama modul seperti app.extractor, app.unit_parser, app.section_pipeline.

Dampak: Pengguna non-teknis (asesor, admin LSP) melihat informasi yang tidak berguna bagi mereka, sementara informasi yang mereka butuhkan (status unit, langkah berikutnya) tenggelam.

Rekomendasi: Sembunyikan detail teknis di balik tautan "Detail teknis" (elemen <details>), atau pindahkan ke halaman Diagnostik. Ganti istilah: "identifikasi: lengkap" → "Semua unit terbaca".

UX-10. Emoji dipakai sebagai ikon antarmuka [P3]

Bukti: Tombol memakai emoji 📚 📦 🖨️ ⚙️ 📥, sementara menu samping memakai glyph ▤ ▦ ▧ ⚙️ ▣. Keduanya dirender berbeda di tiap sistem operasi dan tidak dapat diwarnai.

Dampak: Tampilan terlihat tidak konsisten dan sedikit "mainan" — persis kesan yang membuat aplikasi terasa kurang profesional.

Rekomendasi: Ganti keduanya dengan satu set ikon SVG inline (mis. Lucide/Heroicons, disalin lokal tanpa CDN sesuai prinsip offline aplikasi). Ukuran dan ketebalan garis diseragamkan 20 px / 1,75.

3.3 Yang sudah baik dan sebaiknya dipertahankan

- Tata letak dasar rapi: grid 244 px + konten, kartu bersudut 10 px, bayangan halus, dan tidak ada luapan horizontal di desktop.
- Gaya status konsisten: badge hijau/kuning/merah dipakai seragam untuk status unit, pencocokan, dan regulasi.
- Pesan galat berbahasa Indonesia dan ditampilkan sebagai alert, bukan halaman putih atau traceback.

- Halaman cetak sudah A4 dengan orientasi portrait/landscape otomatis — bagian ini justru paling matang.
- Prinsip tanpa CDN dan tanpa koneksi internet terjaga; aset lokal dan cepat (halaman merespons 10 ms pada data uji).

4. Temuan Keamanan

SEC-01. Hak akses tidak ditegakkan di hampir semua endpoint [P1]

Bukti: Dari 13 endpoint POST, hanya /skkni/{doc_id}/regulasi yang memeriksa peran (app/main.py:221). Endpoint lain — /skkni/upload, /skema/upload, /skema/{id}/muk, /skema/{id}/unit/{kode}/koreksi, /hasil/generate, /skema/{id}/ekstrak — tidak memeriksa sesi maupun peran sama sekali. Seluruh halaman GET juga terbuka tanpa login.

Dampak: Siapa pun yang dapat menjangkau alamat aplikasi (termasuk dari perangkat lain bila bind diubah atau lewat port-forward) dapat mengunggah dokumen, mengubah MUK, mengoreksi unit, dan membuat dokumen. Dokumentasi menyatakan RBAC admin/asesor sudah diterapkan; kenyataannya login hanya dekoratif.

Rekomendasi: Tambahkan dependency tunggal, mis. wajib_login(request, peran_minimal=None), lalu pasang pada setiap route POST dan pada halaman yang mengubah data. Petakan kewenangan: unggah & koreksi & regulasi = admin; susun MUK & cetak = asesor; halaman baca = keduanya. Tambahkan test otomatis yang memastikan setiap POST menolak permintaan tanpa sesi (403).

SEC-02. Kredensial bawaan dibuat otomatis dan ditampilkan di halaman login [P1]

Bukti: app/db.py:123–137 membuat admin/admin123 dan asesor/asesor123 setiap kali tabel pengguna kosong; app/db.py:146 membuat kunci API tetap skkni_live_secret_key_bnsp2023. Halaman login (login.html) mencetak keduanya, dan docs/PANDUAN_ALUR_PENGUJIAN.md memuat ketiganya.

Dampak: Bila basis data dihapus atau dipindahkan, akun dan kunci API kembali ke nilai yang sudah diketahui publik. Kunci API tidak dapat dicabut karena selalu dibuat ulang.

Rekomendasi: Buat kata sandi awal acak, cetak sekali ke konsol/log saat instalasi, dan paksa ganti pada login pertama. Hapus tampilan kredensial dari halaman login. Kunci API default sebaiknya dihapus; buat lewat endpoint admin dengan nilai acak.

SEC-03. API key disimpan sebagai teks biasa tanpa batas waktu [P2]

Bukti: Tabel kunci_api menyimpan kolom kunci apa adanya (app/db.py:106–114); validasi_kunci_api membandingkan string langsung dan menulis timestamp pada setiap permintaan (app/db.py:335–352).

Dampak: Bila data/app.db bocor, seluruh integrasi eksternal dapat dibajak. Penulisan timestamp per permintaan juga menambah beban kunci tulis SQLite.

Rekomendasi: Simpan hash (mis. SHA-256 dengan awalan yang terlihat) dan bandingkan hash-nya. Tambahkan masa berlaku serta cakupan peran. Catat pemakaian terakhir secara asinkron/di memori, bukan tulis per permintaan.

SEC-04. Tidak ada pembatasan percobaan login [P2]

Bukti: proses_login (app/main.py:133–152) tidak memiliki penghitung gagal, jeda, atau captcha.

Dampak: Percobaan kata sandi berulang dapat dilakukan tanpa hambatan; dengan kata sandi bawaan yang pendek, ini cepat berhasil.

Rekomendasi: Tambahkan pembatasan sederhana per IP+username (mis. 5 gagal → tunggu 60 detik) dengan penyimpanan di memori atau tabel. Catat kegagalan di log.

SEC-05. Header keamanan peramban tidak dipasang [P2]

Bukti: Tidak ada middleware yang menambahkan Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, atau Referrer-Policy. /docs, /redoc, dan /openapi.json terbuka tanpa autentikasi.

Dampak: Halaman rentan dibingkai (clickjacking) dan dokumentasi API membeberkan seluruh permukaan endpoint. Risiko rendah selama hanya 127.0.0.1, tetapi menjadi nyata bila dijalankan di jaringan LSP.

Rekomendasi: Tambahkan middleware header (CSP minimal: default-src self; tanpa inline style/script — konsisten dengan perbaikan UX-08). Nonaktifkan docs_url/openapi_url atau lindungi dengan autentikasi saat mode produksi.

SEC-06. Cookie sesi dan CSRF belum diperketat [P3]

Bukti: Cookie skkni_session diset httponly + samesite=lax tanpa secure (app/main.py:151). Token CSRF disimpan di cookie terpisah dan tidak diikat ke sesi; berlaku 8 jam tanpa rotasi (app/api/security.py:18–29).

Dampak: Pada pemakaian http lokal dampaknya kecil, tetapi pola ini tidak aman bila nanti dilayani lewat HTTPS/proxy.

Rekomendasi: Set secure=True saat skema https, tambahkan rotasi token setelah login, dan ikat token CSRF ke id sesi. Pertimbangkan menyatukan token CSRF ke dalam sesi server.

SEC-07. Penanganan unggahan dan path sudah baik (catatan positif) [P3]

Bukti: Nama berkas internal berupa UUID, ID divalidasi regex ^[0-9a-f]{32}$, PDF diperiksa signature + %%EOF + dapat dibuka + tidak terenkripsi, duplikat ditolak lewat SHA-256, ukuran dibatasi 50 MB. Seluruh kueri SQL memakai parameter.

Dampak: Permukaan serangan path traversal dan SQL injection tertutup dengan baik.

Rekomendasi: Pertahankan. Tambahkan pemindaian isi PDF (mis. batas jumlah halaman) agar unggahan raksasa tidak menghabiskan CPU saat identifikasi.

5. Keandalan dan Performa

REL-01. SQLite tanpa mode WAL, padahal ada penulisan paralel [P1]

Bukti: Koneksi dibuka tanpa PRAGMA journal_mode=WAL (app/db.py:29–38). Job ekstraksi berjalan pada ThreadPoolExecutor 2 worker (app/jobs.py:34) sementara permintaan web juga menulis (sesi, timestamp API key, MUK).

Dampak: Pada pemakaian bersamaan, penulisan dapat gagal dengan "database is locked". Gejalanya intermiten dan sulit direproduksi — jenis bug yang paling merepotkan saat demo.

Rekomendasi: Aktifkan WAL dan busy_timeout (mis. 5000 ms) saat membuka koneksi. Ini satu baris perubahan dengan dampak besar.

REL-02. Halaman daftar SKKNI menjalankan identifikasi PDF saat dibuka [P2]

Bukti: app/main.py:181–183 memanggil library.identifikasi(d) di dalam loop untuk setiap dokumen yang belum berstatus, pada setiap kunjungan halaman /skkni.

Dampak: Waktu buka halaman tumbuh linear dengan jumlah dan ukuran PDF. Dengan 5 dokumen saat ini hanya 11 ms, tetapi pada 30 dokumen besar ini bisa menjadi beberapa detik dan memblokir request.

Rekomendasi: Jadikan identifikasi sebagai job latar (mekanisme app/jobs.py sudah ada) dan tampilkan status "sedang diidentifikasi" di tabel. Minimal: batasi satu dokumen per request.

REL-03. Pembacaan JSON berulang tanpa cache (pola N+1) [P2]

Bukti: status_units.status_unit dipanggil per unit per request (main.py:316, 804, 858) dan membaca berkas JSON tiap kali; peta_kelengkapan() juga dihitung ulang setiap halaman detail skema.

Dampak: Setiap muat halaman melakukan puluhan baca berkas. Dampaknya belum terasa karena data masih kecil, tetapi menjadi beban I/O nyata saat satu skema berisi 20–30 unit.

Rekomendasi: Tambahkan cache dalam memori ber-TTL berdasarkan mtime berkas, atau simpan ringkasan status di SQLite saat ekstraksi/koreksi selesai.

REL-04. Pembuatan dokumen Word berjalan di dalam request [P2]

Bukti: POST /hasil/generate memanggil document_service.buat_dokumen secara sinkron (main.py:1086–1106); tidak ada progres maupun batas waktu.

Dampak: Untuk 25 formulir, pengguna menunggu tanpa umpan balik dan koneksi dapat terputus di tengah.

Rekomendasi: Pindahkan ke job manager seperti ekstraksi, lalu arahkan pengguna ke halaman progres. Sekaligus menyelesaikan UX-04.

REL-05. Tidak ada migrasi skema basis data [P3]

Bukti: Tabel dibuat dengan CREATE TABLE IF NOT EXISTS (app/db.py:63–115). Tidak ada tabel versi skema atau langkah ALTER.

Dampak: Penambahan kolom di versi berikutnya tidak akan diterapkan pada basis data lama; aplikasi bisa gagal dengan galat yang membingungkan.

Rekomendasi: Tambahkan kolom versi_skema (PRAGMA user_version) dan urutan migrasi idempoten yang dijalankan saat start.

REL-06. Pencadangan tersedia tetapi tidak terjadwal [P3]

Bukti: tools/backup.py dan tools/restore.py ada, tetapi tidak dipanggil otomatis, tidak tercantum di run.sh, dan tidak tercakup test (test_backup_restore.py berbentuk skrip manual).

Dampak: Kehilangan data hanya terhindar bila pengguna ingat menjalankan pencadangan sendiri.

Rekomendasi: Jalankan pencadangan otomatis harian saat start (rotasi 7 berkas) dan tampilkan tanggal pencadangan terakhir di dashboard.

6. Kualitas Kode dan Arsitektur

CODE-01. Berkas monolit [P2]

Bukti: app/main.py 1.146 baris (32 route halaman + logika render + penanganan galat), app/buku_kerja.py 1.193 baris, app/templates/skema_muk.html 1.365 baris, cetak_apl.html 903 baris, skema_detail.html 687 baris.

Dampak: Setiap perubahan kecil menyentuh berkas raksasa; sulit ditinjau, mudah bentrok bila dua programmer bekerja bersamaan.

Rekomendasi: Pecah main.py per domain (routes/skkni.py, routes/skema.py, routes/muk.py, routes/cetak.py) dengan APIRouter — pola yang sudah dipakai di app/api/. Pecah skema_muk.html menjadi partial per instrumen di app/templates/muk/.

CODE-02. Duplikasi blok yang sama di banyak tempat [P2]

Bukti: Pola render + set_cookie_token diulang di 10+ handler. Blok "gerbang dokumen" (cek 409 + pesan penahan) disalin identik empat kali (main.py:561–571, 615–625, 668–678, dan di API). Definisi ROOT muncul di lima modul (db.py, library.py, jobs.py, document_service.py, buku_kerja.py).

Dampak: Perbaikan harus diulang di semua salinan; risiko salah satu terlewat tinggi.

Rekomendasi: Buat helper tunggal: render_halaman(), cek_gerbang(schema_id), dan satu modul app/paths.py berisi ROOT beserta seluruh direktori data.

CODE-03. Penanganan galat yang menelan kesalahan [P3]

Bukti: app/db.py:202–208: pemeriksaan masa berlaku sesi dibungkus try/except Exception: pass.

Dampak: Sesi dengan format tanggal rusak akan dianggap berlaku selamanya, tanpa jejak di log.

Rekomendasi: Ganti dengan logging peringatan + hapus sesi yang tidak dapat diurai.

CODE-04. Dependensi tidak terpakai dan pin tidak konsisten [P2]

Bukti: requirements.txt memuat flatbuffers, networkx, numpy, onnxruntime, packaging, protobuf, psutil, pymupdf4llm, pymupdf_layout, PyYAML, dan tabulate — tidak satu pun diimpor oleh app/ atau tools/. Ukuran .venv 332 MB. Sebagian paket dipin ==, sebagian >=.

Dampak: Instalasi lambat dan gagal di jaringan terbatas, ukuran paket distribusi membengkak, dan permukaan kerentanan bertambah tanpa manfaat. Build tidak reproducible karena campuran pin.

Rekomendasi: Hapus 11 paket tak terpakai (setelah memastikan tidak ada impor dinamis), lalu pin seluruh dependensi langsung ke versi pasti. Sertakan berkas lock (pip freeze) untuk rilis.

CODE-05. Tidak ada alat kualitas otomatis [P3]

Bukti: Tidak ada konfigurasi ruff/flake8, black, atau mypy; tidak ada pre-commit; tidak ada CI.

Dampak: Standar kode bergantung pada disiplin manual, sehingga gaya penulisan bercampur (mis. dict | None vs Dict[str, Any]).

Rekomendasi: Tambahkan ruff + black dengan konfigurasi minimal, jalankan pada pre-commit, dan pasang GitHub Actions yang menjalankan lint + test.

7. Pengujian

TEST-01. Test suite tidak dapat dijalankan otomatis [P1]

Bukti: Dari 29 berkas di tests/, hanya test_api_v1.py, test_buku_kerja.py, dan test_fase1_auth_db.py yang memakai unittest.TestCase. Perintah "python -m unittest discover -s tests" hanya menjalankan 26 test dan melaporkan OK — 26 berkas lain (termasuk test_web.py, test_document_service.py, test_qa_e2e.py) tidak ikut berjalan. README meminta pengguna menjalankan tiap berkas manual.

Dampak: Regresi pada ekstraksi, DOCX, dan halaman web tidak akan tertangkap otomatis. Laporan "26 test lulus" memberi rasa aman yang menyesatkan.

Rekomendasi: Ubah 26 berkas skrip menjadi fungsi test pytest (kebanyakan hanya perlu membungkus assert dalam def test_*()), tambahkan pytest ke requirements, dan sediakan satu perintah: pytest -q. Tambahkan penanda (marker) untuk test lambat seperti test_cross_skkni.

TEST-02. Tidak ada pengujian hak akses, aksesibilitas, atau UI nyata [P2]

Bukti: Seluruh test halaman memakai TestClient dan memeriksa string HTML. Tidak ada test yang mencoba POST tanpa sesi (karena otorisasi belum ada), tidak ada pemeriksaan kontras/label, dan tidak ada test peramban.

Dampak: Celah SEC-01 tidak akan terdeteksi oleh test mana pun. Masalah tampilan juga tidak terpantau.

Rekomendasi: Tambahkan test negatif otorisasi (semua POST tanpa sesi → 403), lalu satu test peramban tipis (Playwright) untuk alur utama: login → unggah → ekstrak → simpan MUK → unduh DOCX.

TEST-03. Test bergantung pada data lokal, bukan fixture [P3]

Bukti: Test membaca data/pdf/, data/schema_library/, dan data/json/ yang ada di repositori dan berubah selama pemakaian (mis. data/app.db 76 KB berisi sesi nyata).

Dampak: Hasil test berbeda antar mesin dan setelah pemakaian normal, sehingga kegagalan sulit dibedakan antara bug dan data.

Rekomendasi: Sediakan direktori fixture tetap + variabel lingkungan untuk mengarahkan ROOT data saat test. Jangan sertakan app.db di repositori.

8. Dokumentasi dan Konsistensi Klaim

DOC-01. Status di dokumentasi sudah usang [P2]

Bukti: README.md menandai Fase 4F "belum", padahal docs/CATATAN_PERBAIKAN_FASE_4F.md menyatakan selesai. docs/RENCANA_PENGEMBANGAN_BUKU_KERJA.md menandai Fase 4 (REST API) "BELUM", padahal commit b6ce013 sudah mengimplementasikannya dan test_api_v1.py lulus.

Dampak: Programmer baru atau vendor luar akan salah menilai kesiapan aplikasi dan mungkin mengerjakan ulang fitur yang sudah ada.

Rekomendasi: Perbarui kedua tabel status, lalu jadikan satu berkas sebagai sumber kebenaran (mis. docs/STATUS.md) dan tautkan dari README.

DOC-02. Klaim fitur melebihi kenyataan [P2]

Bukti: Catatan Fase 4F menyebut "Editor koreksi per-unit berdampingan PDF + SKKNI"; kenyataannya halaman unit_koreksi.html hanya menyediakan tautan "Buka PDF hal. X" yang membuka tab baru (baris 19–20). Klaim "pencegahan kehilangan data" juga tidak ada (lihat UX-03).

Dampak: Penguji menerima laporan yang tidak sesuai produk, sehingga temuan menjadi tidak dapat dipercaya saat audit berikutnya.

Rekomendasi: Tulis ulang klaim sesuai perilaku nyata, atau implementasikan panel PDF tersemat (iframe #page=N) agar klaim menjadi benar.

DOC-03. Delapan dokumen tanpa indeks dan sebagian tumpang tindih [P3]

Bukti: docs/ memuat INSTALL, USER_GUIDE, PANDUAN_ALUR_PENGUJIAN, BACKUP_RESTORE, RELEASE_NOTES_1.0.0, CATATAN_PERBAIKAN_FASE_4F, RENCANA_PENGEMBANGAN, dan README akar — tanpa halaman indeks. Kredensial dan kunci API ditulis di PANDUAN_ALUR_PENGUJIAN.

Dampak: Pembaca tidak tahu harus mulai dari mana; kredensial tersebar di berkas yang mudah dibagikan.

Rekomendasi: Buat docs/README.md sebagai indeks (instalasi → panduan pengguna → panduan uji → catatan rilis). Pindahkan kredensial ke berkas instalasi yang tidak ikut dibagikan.

9. Prioritas Perbaikan untuk Programmer

Urutan di bawah disusun berdasarkan risiko, bukan kemudahan. Tiga butir pertama sebaiknya selesai sebelum aplikasi dipakai lebih dari satu orang.

Tahap 1 — Wajib sebelum dipakai bersama (1 minggu)

No	Pekerjaan	Rujukan	Ukuran selesai
1	Tegakkan login + peran pada semua POST dan halaman ubah data	SEC-01	Test: semua POST tanpa sesi → 403
2	Hapus kredensial bawaan & tampilan kredensial di halaman login	SEC-02	Tidak ada kata sandi di UI/dokumen
3	Aktifkan WAL + busy_timeout SQLite	REL-01	Uji 2 job bersamaan tanpa "database is locked"
4	Perbaiki kontras teks sekunder & badge; perbesar target klik	UX-02, UX-06	Semua teks ≥ 4,5:1; tombol ≥ 32 px
5	Migrasikan test ke pytest + satu perintah jalan	TEST-01	pytest -q menjalankan seluruh berkas

Tahap 2 — Tampilan dan kenyamanan (2 minggu)

No	Pekerjaan	Rujukan	Ukuran selesai
6	Satu sistem desain: hapus 1.416 style inline, buat komponen di style.css	UX-08, CODE-01	Tidak ada style= di template aplikasi
7	Pecah halaman MUK per instrumen + penanda belum tersimpan + beforeunload	UX-03	Tidak ada data hilang saat tab ditutup
8	Indikator proses pada unggah, generate DOCX, dan ekstraksi	UX-04, REL-04	Tombol nonaktif + progres terlihat
9	Label form, fokus terlihat, ikon SVG, navigasi ponsel	UX-05, UX-07, UX-10	Lighthouse aksesibilitas ≥ 90
10	Blok "Langkah berikutnya" + aksi utama berwarna di tiap halaman	UX-01	Satu aksi utama per halaman
11	Sembunyikan istilah teknis di balik "Detail teknis"	UX-09	Halaman detail bebas path/SHA

Tahap 3 — Perawatan jangka panjang (2 minggu)

No	Pekerjaan	Rujukan	Ukuran selesai
12	Pecah main.py & template besar; satukan helper duplikat	CODE-01, CODE-02	Tidak ada berkas > 500 baris
13	Hapus 11 dependensi tak terpakai, pin semua versi	CODE-04	pip install < 100 MB

14	Cache status unit + identifikasi sebagai job latar	REL-02, REL-03	Halaman detail < 100 ms di 30 unit
15	Hash API key, batasi percobaan login, header keamanan	SEC-03–05	Lulus uji negatif keamanan
16	Selaraskan dokumentasi dengan kenyataan	DOC-01–03	Status & klaim sesuai perilaku
17	Ruff + black + CI; test peramban alur utama	CODE-05, TEST-02	CI hijau pada setiap PR

10. Lampiran: Data Pendukung

A. Berkas terbesar

Berkas	Baris	Catatan
app/templates/skema_muk.html	1.365	353 atribut style inline, 88 kontrol, 9 tab
app/buku_kerja.py	1.193	Penyusunan data 25 dokumen + konfigurasi MUK
app/main.py	1.146	32 route halaman, logika render, penanganan galat
app/templates/cetak_apl.html	903	CSS cetak 903 baris terpisah dari style.css
app/templates/skema_detail.html	687	95 atribut style inline
app/buku_kerja_docx.py	551	Generator DOCX/ZIP 25 formulir
app/templates/unit_koreksi.html	428	60 atribut style inline
app/api/v1.py	461	REST API eksternal dengan API key

B. Hasil pengukuran antarmuka

Yang diukur	Hasil	Ambang / standar
Kontras teks .muted / footer	4,02 : 1	≥ 4,5 : 1 (WCAG AA)
Kontras .stat-label, header tabel	4,39 : 1 pada 11,5–12,5 px	≥ 4,5 : 1
Kontras badge hijau	3,49 : 1	≥ 4,5 : 1
Tinggi tombol aksi tabel	24–27 px	≥ 32 px (disarankan 44)
Label form terhubung (for/id)	1 dari 102 kontrol	100%
Atribut aria/role di template	3	sesuai kebutuhan pembaca layar
Atribut style inline	1.416	0 (pakai kelas)
Luapan menu di 390 px	bilah 946 px, harus digeser	tanpa geser horizontal
Waktu muat halaman (data uji)	8–11 ms	baik
Test yang berjalan via discover	26 dari ±28 berkas	seluruh berkas

C. Cara mereproduksi pengukuran

1. Jalankan aplikasi: ./run.sh (default http://127.0.0.1:8000).
2. Buka DevTools → Console, lalu periksa getComputedStyle(document.querySelector(".muted")).color dan latar kartu untuk menghitung kontras.
3. Ukur target klik: [...document.querySelectorAll("a,button")].map(e => e.getBoundingClientRect().height).
4. Uji ponsel: DevTools → Toggle device toolbar → iPhone 12 (390×844).
5. Hitung gaya inline: grep -o 'style="' app/templates/**/*.html | wc -l.
6. Jalankan test: .venv/bin/python -m unittest discover -s tests -v (perhatikan jumlah test yang berjalan).
7. Periksa status dokumentasi: bandingkan README tabel Fase 4F dengan docs/CATATAN_PERBAIKAN_FASE_4F.md.

D. Catatan penutup

Audit ini menilai kode dan antarmuka sebagaimana adanya pada commit b2a12a5, tanpa mengubah apa pun. Semua angka pada laporan berasal dari pengukuran langsung atau pembacaan kode; tidak ada temuan yang disimpulkan dari kesan visual saja. Bila ada butir yang perlu diperdebatkan, setiap temuan sudah menyertakan lokasi berkas agar dapat diverifikasi ulang.

Satu catatan jujur: aplikasi ini lebih matang daripada kesan yang ditimbulkannya. Sebagian besar keluhan "kurang menarik" dapat diselesaikan tanpa menulis ulang fitur — cukup menata ulang hierarki, menaikkan kontras, menyeragamkan tombol, dan memecah satu halaman yang terlalu padat.